Git Lab

Program 1: Setting Up and Basic Commands

Initialize a new Git repository in a directory. Create a new file and add it to the staging area and commit the changes with an appropriate commit message

1. Create a Git Project Folder

→ mkdir gitlab
→ cd gitlab

Creates and enters the project folder.

2. Create Experiment Folder

→ mkdir exp1
→ cd exp1

Creates and enters the experiment folder.

3. Initialize Git

→ git init

Creates a new Git repository.

4. Create a File

→ echo Hello Git > file.txt

Creates file.txt and adds content to it.

5. Check Files

→ ls

Displays the files in the current folder.

6. Add File to Staging

→ git add .

Adds the file to the staging area for committing.

7. Create First Commit

→ git commit -m "test commit"

Saves the changes as the first commit.

8. Make a Second Change

Modify file.txt by adding some new content.

9. Add the Updated File

→ git add file.txt

Stages the new changes.

10. Create Second Commit

→ git commit -m "second commit"

Saves the second set of changes as a new commit.

11. View Commit History

→ git log

Shows detailed information about previous commits.

12. View Short Commit History

→ git log --oneline

Shows commits in a short format with their commit hash and message.

Example:

0fe34f7 second commit

ea34902 test commit

Hash → Unique ID of the commit
Commit message → Description of what was changed

Program 2: Creating and Managing Branches

Create a new branch named "feature-branch." Switch to the "master" branch. Merge the "feature-branch" into "master."

Step 1: Open Git Bash and Navigate to the Repository

→ cd gitlab/exp1

Moves into the existing Git repository (exp1).

Step 2: Check Repository Status

→ git status

Checks the current branch and whether there are any pending changes.

Step 3: Create a Feature Branch

→ git branch feature_branch

Creates a new branch named feature_branch.

Step 4: View All Branches

→ git branch

Displays all available branches.

Example:

feature_branch

* master

(* Indicates the branch currently in use.)

Step 5: Switch to the Feature Branch

→ git checkout feature_branch

Switches from master to feature_branch.

Step 6: Make Changes in the Feature Branch

→ echo "Feature added" >> file.txt

Adds new content to file.txt.

Step 7: Add the Changes

→ git add .

Adds the changes to the staging area.

Step 8: Commit the Changes

→ git commit -m "added feature"

Saves the changes as a commit in feature_branch.

Step 9: Switch Back to Master

→ git checkout master

Switches back to the master branch.

Step 10: Merge the Feature Branch

→ git merge feature_branch

Merges the changes from feature_branch into master.

Step 11: Check the Current Branch

→ git checkout master

Ensures that we are on the master branch.

Step 12: Try Merging Again

→ git merge feature_branch

Output:

Already up to date.

Means the changes from feature_branch have already been merged.

Step 13: View Commit History

→ git log –oneline

git branch → Create a branch
git checkout → Switch branch
git add → Stage changes
git commit → Save changes
git merge → Combine branch changes
git log → View commit history